

Judges Said — LinkedIn Video Script

Full-Stack Generative AI · Hallucination-Resistant RAG for German Case Law

TARGET 2:45 · ~420 WORDS NARRATION · ENGLISH

1 Intro

~15s

SPEAK — CAMERA

Hi, I'm Bechir. I want to show you something I built and deployed: a search engine for German labour-law court decisions.

But the interesting part isn't the search. It's what happens when it **refuses** to answer.

2 The problem

~25s

SPEAK — CAMERA

In Germany, court decisions are public domain. By law, nobody owns them.

But if you want to actually search them, you need a Beck-online or juris subscription. So the people who need this most — employees who just got fired, small-business HR, works councils — are exactly the people who can't afford it.

So I built an open alternative. You describe your situation in plain English, and you get the real decisions that match.

3 Demo 1 — it works

~30s

SCREEN RECORDING

TYPE:

my employer fired me during my probation period while I was on sick leave

I'm asking in plain English — no legal terms, just my situation.

And I get back real decisions. Arbeitsgericht Aachen. The Aktenzeichen. The date. The exact paragraphs the ruling turns on.

CLICK: a citation chip, to open the source passage.

Every claim links to the passage it came from. I can read the original German the answer is based on.

4 Hallucination

~35s

SPEAK — CAMERA

Now here's the problem with putting an LLM anywhere near case law.

Ask a language model for a court decision, and it will happily invent one. A real-looking court, a real-looking case number — completely fake. In legal work, that's the worst possible failure: confident, specific, and wrong.

So in this system, the model is never trusted on its own.

Before any answer reaches you, a validator checks it. Every quote must appear **word for word** in the source. Every citation must come from a document that retrieval actually returned this turn. If a check fails, the answer is regenerated once — and if it fails again, you get a refusal instead of a guess.

And that validator is plain code. No second AI grading the first. It's deterministic, and it's unit-tested — which a prompt can never be.

5 Demo 2 — it refuses

~25s

SCREEN RECORDING

TYPE:

what are my chances of winning?

Watch this. I'm asking it to predict my chances.

SHOW: the refusal. Hold the pause — let it land.

It refuses. And this isn't politeness — it's the law.

Under the German RDG, showing you what courts decided is *information*. Telling you what will happen in *your* case is regulated legal advice. So that boundary is enforced in code, before the answer is ever shown. The system stays on the right side of the line by design.

6 Technical

~50s

SPEAK — CAMERA OR SLIDES

Quickly, how it works underneath.

Retrieval is hybrid. Two searches run in parallel — semantic search over vector embeddings using pgvector with an HNSW index, and keyword search using Postgres full-text search with the German dictionary. That German config matters: German compounds and stemming mean the word "Kündigungen" matches six times more passages under German than under English.

The two result sets get fused with **Reciprocal Rank Fusion** — forty candidates from each side, merged by rank, capped at two passages per decision so a single long judgment can't fill the whole answer.

The corpus is 1,070 court decisions and 5,680 statute sections — about 41,000 passages. They're chunked at 350 tokens with 50 overlap, never across a section boundary, so a citation always points at a real part of the ruling: Tenor, Tatbestand, Entscheidungsgründe.

Embeddings run locally — multilingual E5, 768 dimensions. Nothing gets sent to an embedding API. And because the model is multilingual, you can ask in English and still match German passages.

Backend is FastAPI in Docker on a VPS behind nginx. Frontend is React. Postgres and auth on Supabase.

SPEAK — CAMERA

The thing I'd take away from this project: making a RAG system that answers is easy. Making one that **knows when to shut up** is the hard part — and in a regulated domain, it's the only part that matters.

Code and live demo are in the comments. Happy to talk about any of it.

DELIVERY NOTES

Pacing: ~150 words per minute. If you run long, cut the corpus paragraph in section 6 — it is the least load-bearing.

The hook is section 4 → 5. Most demo videos show the thing working. Yours shows it *refusing*, and explains why refusing is the feature. Do not rush the pause after the refusal appears.

Say "Aktenzeichen", "Arbeitsgericht", "Kündigungen" in German. It signals you know the domain, not just the stack.

Post text: open with the tension, not the tech — for example: "I built a legal search engine. The best thing it does is refuse to answer."